

ECoG-Based Finger Flexion Prediction Using Ensemble Learning and Feature-Driven Smoothing

BE 5210 Final Project Report

TEAM NAME: DEF CALCULATE FEATURE WINDOWS

Jadd El Hussein, Juee Naik, Samhitha Vedire

Abstract—Decoding finger movements from electrocorticography (ECoG) signals is a critical task in the development of brain-computer interfaces (BCIs) and neuroprosthetic devices. This paper presents a feature-driven machine learning framework for predicting continuous finger flexion trajectories from ECoG recordings. We extract rich time-domain and high-gamma frequency features from segmented ECoG data, selecting feature domains known to correlate with motor execution and planning. An ensemble of XGBoost regressors, each tuned for strength and regularization, is trained for individual finger predictions and subsequently integrated through a RidgeCV meta-learner to promote generalization and prevent overfitting. Post-processing techniques, including interpolation and adaptive Savitzky-Golay smoothing, further refine the decoded trajectories to match the temporal dynamics of natural finger movements. The proposed pipeline achieves high correlations between predicted and true finger positions across multiple subjects, demonstrating its effectiveness and practical suitability.

I. INTRODUCTION

The ability to decode intended motor actions from neural signals is a fundamental challenge in brain-computer interface (BCI) research. Among the various recording modalities, electrocorticography (ECoG) offers a favorable compromise between invasiveness and signal quality, providing high temporal and spatial resolution suitable for continuous motor decoding. Accurate prediction of finger flexions from ECoG data has broad implications for the development of neuroprosthetic limbs, rehabilitation devices, and assistive technologies for individuals with motor impairments.

Despite its promise, decoding finger trajectories from ECoG signals remains difficult. Neural signals are noisy, high-dimensional, and exhibit nonstationary properties over time. Additionally, the cortical representations of finger movements are overlapping and nonlinear, making simple linear models insufficient for achieving high-fidelity predictions. Furthermore, real-world applications demand decoding algorithms that are both accurate and computationally efficient, capable of generalizing across sessions and subjects.

To address these challenges, we propose a machine learning framework that combines structured feature extraction with ensemble-based regression. Our approach focuses on extracting physiologically meaningful time-domain and high-gamma frequency features from windowed ECoG segments, preserving both local dynamics and broader cortical activity patterns. We train an ensemble of gradient-boosted decision tree models (XGBoost) for each finger, aggregating

their predictions through a RidgeCV meta-learner to enhance robustness against overfitting. Finally, adaptive post-processing techniques, including one-dimensional interpolation and variance-dependent smoothing, are applied to align the decoded signals more closely with natural finger movement dynamics.

By combining these components into a unified framework, we provide an effective approach for ECoG-based motor decoding that achieves high predictive accuracy and consistent performance across diverse subjects.

II. FINAL ALGORITHM

We developed a machine learning framework to predict continuous finger flexion trajectories from ECoG signals. Raw ECoG recordings were segmented into overlapping windows, and a set of time-domain and high-gamma frequency features were extracted from each window. These feature vectors were used to train an ensemble of three XGBoost regressors for each individual finger, with predictions combined through a RidgeCV meta-learner to enhance stability and accuracy. After model inference, the predicted flexion trajectories were upsampled using one-dimensional interpolation and adaptively smoothed based on signal variance to better match natural movement dynamics. Final performance was evaluated by computing Pearson correlation coefficients between the predicted and true finger trajectories across multiple subjects. The overall flow of the algorithm is given in Figure 1.

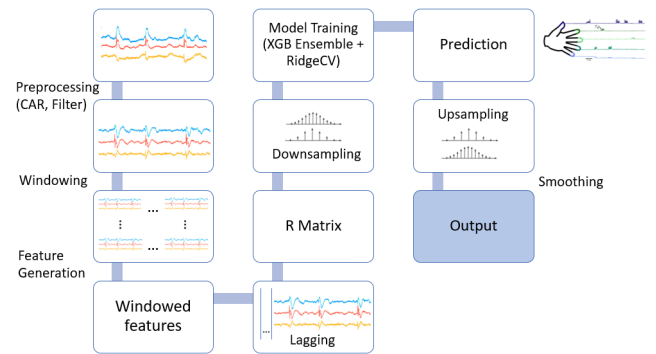


Fig. 1. Steps of the Prediction Pipeline

The details of our algorithm are discussed as follows:

A. Pre-Processing

We first apply a common average reference (CAR) to the raw ECoG data to reduce spatially correlated noise across channels. Next, we apply a zero-phase Butterworth bandpass filter (0.1–200 Hz) of order 4 and a 60 Hz notch filter to remove line noise and slow drifts. Each channel is then normalized to have zero mean and unit variance. The preprocessed signal is segmented into overlapping windows, with window length 0.1s and overlap 0.05s, chosen to balance temporal resolution and feature stability. The dataglove and ECoG data are temporally aligned by downsampling the glove data to match the number of ECoG feature windows, with a small causal shift (37 ms) to account for neural-motor delay.

B. Feature Generation

For each window and channel, we extract a set of time- and frequency-domain features. These include: *mean*, *standard deviation*, *signal derivative*, *spectral entropy*, *local motor potential (LMP)*, and *bandpower in multiple high-gamma bands (75–115 Hz, 125–159 Hz, 159–175 Hz)*. These features are concatenated across channels to form a feature vector for each window. To capture temporal context, we stack features from the current and 4 previous windows, forming an evolution matrix as described by Kubánek et al. This approach allows the model to access information from a 400 ms history for each prediction. The formulae below show the features extracted from each window and channel.

1) Mean

$$\mu(x) = \frac{1}{N} \sum_{i=0}^{N-1} x_i \quad (1)$$

2) Standard Deviation

$$\sigma(x) = \sqrt{\frac{1}{N-1} \sum_{i=0}^{N-1} (x_i - \mu)^2} \quad (2)$$

3) Signal Derivative

$$\dot{x}(t) = \mu[x[n] - x[n-1]] \quad (3)$$

4) Spectral Entropy

$$H_s = - \sum_f P(f) \log_2 [P(f)] \quad (4)$$

5) Local Motor Potential

$$LMP(x) = \frac{1}{N} \sum_{i=-\frac{N}{2}}^{\frac{N}{2}} x_{t+i} \quad (5)$$

C. Ensemble Regression

To predict finger flexion in the 3 subjects, we employ an ensemble learning strategy. For each finger, we train 3 XGBoost regressors, each with differently tuned hyperparameters, and a meta-level RidgeCV regression model to combine their predictions. This stacked ensemble approach leverages the strengths of both tree-based and linear models, improving generalization. RidgeCV inherently incorporates

cross-validation to prevent major overfitting. Models are trained with early stopping and regularization to further reduce overfitting. Validation is performed using a temporal 95:5 split, which allows for an assessment of model performance while utilizing the majority of the data, ensuring more robust training.

D. Post-Processing

The ensemble model generates predictions at the windowed feature rate, which is lower than the original ECoG sampling rate. We upsample the predictions to the full timebase using Akima1D interpolation, followed by adaptive Savitzky-Golay smoothing to reduce residual noise. The final output is reshaped and stored as a .mat file, with each subject's predictions stored as a $147,500 \times 5$ matrix.

III. KEY FIGURES AND MOTIVATIONS

Plots comparing ground truth and predicted finger flexions allowed us to directly assess model performance beyond correlation scores. In Figures 2 and 3 below, we plot data comparing ground truth and predicted finger flexions for one subject. By analyzing mismatches in timing, amplitude, or smoothness, we identified areas where our feature extraction or post-processing needed refinement. For example, observing jagged predictions led us to implement Akima1D interpolation and adaptive smoothing, while consistent amplitude errors motivated tuning model regularization. These visual evaluations were critical for diagnosing model behavior and guiding iterative improvements.

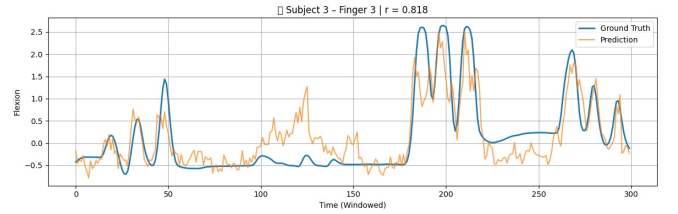


Fig. 2. Subject 3 - Finger 3 Ground Truth vs. Predictions

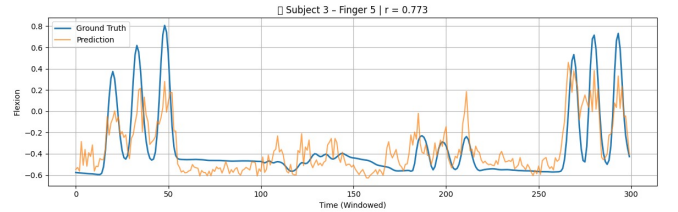


Fig. 3. Subject 3 - Finger 5 Ground Truth vs. Predictions

IV. ALTERNATIVE METHODS ATTEMPTED

A. Spectral Features Across Too Many Bands

Including features from low and mid-frequency bands (e.g., alpha, beta) introduced noise rather than improving decoding. These bands were less informative for finger movement decoding, and their inclusion degraded model

performance. The decision to remove some of the low frequency bands helped increase correlation score as well as improve our run-time.

B. Principal Component Analysis (PCA) for Dimensionality Reduction

PCA was tested to reduce feature space before regression. However, it removed important signal-specific features critical for prediction accuracy, causing a significant drop in performance.

C. Training without Temporal Context (No Lagged Windows)

Models trained without stacking lagged windows performed significantly worse. This validated that including temporal history is critical for capturing the delayed motor responses in ECoG signals.

D. Alternative Prediction Models

In the early stages of the project, we explored several alternative modeling approaches beyond XGBoost and RidgeCV. These included Lasso Regression, Random Forest Regression, Support Vector Regression (SVR), and shallow Multi-Layer Perceptrons (MLPs). Additionally, we experimented with building a small Convolutional Neural Network (CNN) for feature weighting. However, Lasso and simple linear models underfit the nonlinear relationships present in the ECoG signals, while Random Forests and SVRs proved computationally expensive and difficult to tune for optimal performance. The CNN-based approach, although promising in theory, required extensive hyperparameter optimization and carried a high risk of overfitting. Moreover, early evaluations showed that simpler models trained on carefully engineered features could achieve comparable or superior performance with much lower training time. Ultimately, XGBoost ensembles and RidgeCV meta-learning offered a better balance between performance and generalization.

V. DISCUSSION

A. Hyperparameter Tuning

In order to maximize model performance while minimizing overfitting, we carefully tuned the hyperparameters of each XGBoost model. Specific parameters we tuned included the maximum tree depth (*max_depth*), learning rate (*learning_rate*), subsample ratio (*subsample*), and feature column sampling ratio (*colsample_bytree*). We employed different sets of hyperparameters across the three base models per finger to encourage diversity among the models and reduce the risk of all models making similar errors. Early stopping criteria based on validation set performance were also used to avoid unnecessary training beyond optimal convergence. The RidgeCV meta-learner further selected optimal regularization parameters (*alpha*) through cross-validation, ensuring the final stacked predictions were robust and generalized well across unseen data.

B. Ring Finger Flexion Correlation

It is common to drop the fourth (ring) finger from predictions in finger movement decoding tasks because its movements are highly correlated with those of the third and fifth fingers, making it difficult to achieve independent predictions. This correlation arises from both muscular and neural anatomy: the ring finger shares flexor and extensor muscles with the middle and little fingers, particularly through muscles like the lumbricals and the flexor digitorum profundus. In addition, the neural control of the ring and little fingers is intertwined, as both are largely innervated by the ulnar nerve, while the middle finger also receives input from both the radial and ulnar nerves. This shared innervation and muscle structure mean that when the brain sends a signal to move one of these fingers, the others often move as well, resulting in mechanical and neuromuscular coupling. As a result, the ring finger cannot move as independently as the index or thumb, and its flexion is strongly dependent on the activity of its neighboring fingers. For these reasons, including the ring finger in the prediction task can introduce redundancy and reduce the specificity of the decoding model, so it has been excluded to focus on fingers with more independent control.

VI. CONCLUSION

Working on this project allowed us to apply concepts from neuroscience, machine learning, and signal processing in a meaningful way which helped us better understand all the topics learned in class and in our homework. Throughout the project our performance improved steadily through careful feature engineering, model stacking, and strategic signal interpolation. We also learned the importance of rigorous preprocessing, temporal feature stacking, robust validation, and careful post-processing to minimize information loss and refine model outputs. Overall, we are very proud of our final pipeline and the strong decoding performance we achieved across subjects.

VII. REFERENCES

- [1] J. Kubánek, P. Miller, J. Ojemann, E. Wolpaw, and G. Schalk, "Decoding flexion of individual fingers using electrocorticographic signals in humans," *Journal of Neural Engineering*, vol. 6, no. 6, p. 066001, 2009.
- [2] G. Schalk, D. McFarland, T. Hinterberger, N. Birbaumer, and J. Wolpaw, "BCI2000: A general-purpose brain-computer interface (BCI) system," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 6, pp. 1034–1043, 2004.

VIII. APPENDIX

This section contains the final code submitted for the project.

```
!pip install antropy
# =====
# IMPORTS
# =====
import numpy as np
import scipy.io as sio
from tqdm import tqdm
from xgboost import XGBRegressor
from sklearn.multioutput import MultiOutputRegressor
```

```

from scipy.stats import pearsonr, skew
from scipy.signal import butter, filtfilt,
    hilbert, savgol_filter
from sklearn.linear_model import RidgeCV
from scipy.interpolate import
    Akima1DInterpolator
import matplotlib.pyplot as plt
from antropy import spectral_entropy
import os
import pickle
from sklearn.linear_model import RidgeCV
import matplotlib.pyplot as plt

# =====
# LOAD MODELS
# =====
with open("ensemble_models.pkl", "rb") as f:
    ensemble_models = pickle.load(f)

# =====
# FILTER & CREATE FEATURES
# =====
from scipy.signal import butter, filtfilt,
    iirnotch

def apply_car(eeg):
    """Common average referencing (CAR)"""
    return eeg - np.mean(eeg, axis=1, keepdims=
        True)

def filter_data(raw_eeg, fs=1000, lowcut=0.1,
    highcut=200, order=4):
    raw_eeg = apply_car(raw_eeg.astype(np.
        float64))

    # --- Bandpass filter ---
    nyq = 0.5 * fs
    b_band, a_band = butter(order, [lowcut/nyq,
        highcut/nyq], btype='band')
    eeg_bandpassed = filtfilt(b_band, a_band,
        raw_eeg, axis=0)

    # --- 60 Hz Notch filter ---
    notch_freq = 60.0 # Hz
    Q = 40.0 # Quality factor (higher =
        narrower)
    b_notch, a_notch = iirnotch(w0=notch_freq /
        nyq, Q=Q)
    eeg_filtered = filtfilt(b_notch, a_notch,
        eeg_bandpassed, axis=0)

    return eeg_filtered

def lmp(signal, window_len=100):
    """Local Motor Potential: Moving average of
    the signal"""
    return np.convolve(signal, np.ones(
        window_len)/window_len, mode='same')

def hilbert_bandpower(signal, fs, low, high):
    nyq = 0.5 * fs
    b, a = butter(4, [low/nyq, high/nyq], btype=
        'band')
    band = filtfilt(b, a, signal)
    envelope = np.abs(hilbert(band))
    return np.mean(envelope)

def get_features(window, fs=1000):
    # Focused feature set: 6 time-domain + 3
    high-gamma bands
    bands = [(75, 115), (125, 159), (159, 175)]
    num_channels = window.shape[1]
    features = np.zeros((num_channels, 6 + len(
        bands))) # 6 time + 3 freq

```

```

for ch in range(num_channels):
    signal_ch = window[:, ch]
    features[ch, 0] = np.mean(signal_ch)
    # Mean
    features[ch, 1] = np.std(signal_ch)
    # Std
    features[ch, 2] = np.mean(np.diff(
        signal_ch)) # Derivative
    features[ch, 3] = spectral_entropy(
        signal_ch, sf=fs, normalize=True) #
        Entropy
    features[ch, 4] = np.mean(lmp(signal_ch,
        window_len=100)) # LMP

    for i, (low, high) in enumerate(bands):
        features[ch, 5 + i] =
            hilbert_bandpower(signal_ch, fs,
                low, high)

    return features

# Create windows of features
def get_windowed_feats(raw_ecog, fs,
    window_length, window_overlap, lags=[0, 4]):
    """
    Extracts features for each window, including
    time-lagged windows.
    lags: List of relative time steps to include
    (e.g., [0, 1] includes t and t-1)
    """
    window_length = int(window_length * fs)
    window_overlap = int(window_overlap * fs)
    step_size = window_length - window_overlap
    clean_data = filter_data(raw_ecog, fs)

    all_feats = []

    # Precompute features per window
    raw_feats = []
    for start in range(0, clean_data.shape[0] -
        window_length + 1, step_size):
        window = clean_data[start:start +
            window_length, :]
        feats = get_features(window, fs)
        raw_feats.append(feats.flatten())

    raw_feats = np.array(raw_feats)

    # Build feature matrix with time lags
    for i in range(max(lags), len(raw_feats)):
        lagged = [raw_feats[i - l] for l in lags
        ]
        all_feats.append(np.concatenate(lagged))

    return np.array(all_feats)

# Shift the data glove data back by 37 ms and
    downsample it

def downsample(flexion, num_windows, shift_ms
    =37, fs=1000, window_len_ms=100,
    window_overlap_ms=50):
    """Downsample flexion with a -50 ms causal
    shift"""
    shift_samples = int((shift_ms / 1000.0) * fs
        )
    flexion = np.roll(flexion, -shift_samples,
        axis=0)
    effective_fs = fs / (window_len_ms -
        window_overlap_ms)
    downsample_factor = int(fs / effective_fs)
    downsampled = flexion[:, :downsample_factor]
    return downsampled[:num_windows]

# R matrix

```

```

def create_R_matrix(features, N_wind):
    M, num_feats = features.shape
    padded_features = np.vstack([np.tile(
        features[0], (N_wind - 1, 1)), features
    ])
    R = np.ones((M, 1 + N_wind * num_feats))
    for t in range(M):
        R[t, 1:] = padded_features[t:t + N_wind
            ].flatten()
    return R

# Upsample the predictions

def upsample_predictions(y_pred_downsampled,
    original_length, window_length,
    window_overlap, fs=1000, smooth=True):
    y_pred_upsampled = []
    step_size = int((window_length -
        window_overlap) * fs)
    full_time = np.arange(original_length)

    for subject_data in y_pred_downsampled:
        num_windows = subject_data.shape[0]
        time_pred = np.arange(0, num_windows *
            step_size, step_size)
        if time_pred[-1] < original_length - 1:
            time_pred = np.append(time_pred,
                original_length - 1)
            subject_data = np.vstack([
                subject_data, subject_data[-1]])

        # 1D interpolation of data
        upsampled_data = np.vstack([
            Akima1DInterpolator(time_pred,
                subject_data[:, i])(full_time)
            for i in range(subject_data.shape
                [1])
        ]).T

        if smooth:
            for i in range(upsampled_data.shape
                [1]):
                signal = upsampled_data[:, i]

                # Adaptive smoothing: smooth
                # only if variance is low
                local_std = np.std(signal)
                if local_std < 0.02:
                    signal = savgol_filter(
                        signal, window_length
                        =51, polyorder=2, mode='
                        interp')
                else:
                    # Light smoothing to reduce
                    # noise without killing
                    # spikes
                    signal = savgol_filter(
                        signal, window_length
                        =11, polyorder=2, mode='
                        interp')

                upsampled_data[:, i] = signal

            y_pred_upsampled.append(upsampled_data)

    return y_pred_upsampled

# =====
# LOAD DATA
# =====
truetest_data = sio.loadmat('truetest_data.mat')

# =====
# PREDICT ON HIDDEN DATA

```

```

# =====
predicted_dg_test = []

# Parameters
fs = 1000
window_length = 0.1
window_overlap = 0.05
N_wind = 3

for i in tqdm(range(3)):
    ecog_test = truetest_data['truetest_data'][i
        ][0]
    feats_test = get_windowed_feats(ecog_test,
        fs, window_length, window_overlap)
    R_test = create_R_matrix(feats_test, N_wind)
    models = ensemble_models[i]
    Y_pred_downsampled = np.zeros((R_test.shape
        [0], 5))

    for j in tqdm(range(5)):
        base_preds = np.stack([m.predict(R_test)
            for m in models[j]["base_models"]],
            axis=1)
        final_pred = models[j]["meta_model"].
            predict(base_preds)
        Y_pred_downsampled[:, j] = final_pred

    Y_pred = upsample_predictions([
        Y_pred_downsampled], ecog_test.shape[0],
        window_length, window_overlap)[0]
    predicted_dg_test.append(Y_pred)

# Save final predictions
matlab_cell = np.zeros((3, 1), dtype=object)
for i in range(3):
    matlab_cell[i, 0] = predicted_dg_test[i]

sio.savemat('predictions.mat', {'predicted_dg':
    matlab_cell},
    long_field_names=True, do_compression=
    True)

```